

第五章—中间代码生成

编译程序的目标是将源程序翻译成等价的目标程序，为了得到高效目标代码，有时会先生成中间代码。将高级语言翻译成中间代码需要完成哪些工作呢？



- 1 中间代码的形式
- 2 语言的含义（语义）如何描述
- 3 翻译的方法

回顾

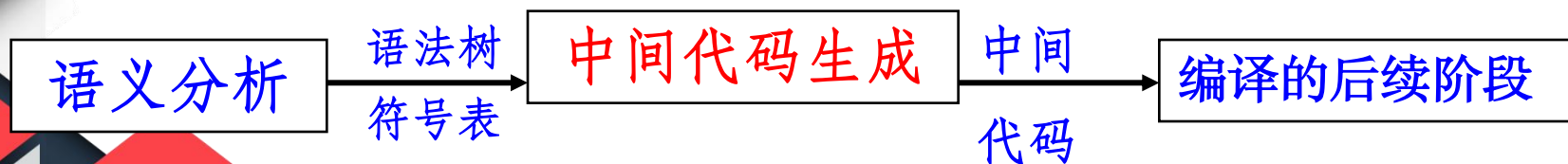
静态语义检查：类型、控制流、一致性、相关名字等的检查

- 类型检查。主要检查操作数和操作符是否相容。
- 控制流检查。用以保证控制语句有合法的转向点。
- 一致性检查。多数语言要求对象只能被定义一次。
- 相关名字检查。有的语言同一名字必须出现两次或者多次。

5.1 中间代码生成概述

为什么要生成中间代码？

- 逻辑结构清楚；
- 利于不同目标机上实现同一种语言；
- 有利于进行与机器无关的优化，这些内部形式也能用于解释。



生成中间代码是为了缩小高级语言和机器语言之间的鸿沟，可以设计多层中间语言。

5.2 中间代码

- 中间代码是源程序的一种内部表示，不依赖目标机的结构，易于机械生成目标代码的中间表示。

中间代码的几种形式

- 逆波兰
- 三地址代码（三元式、四元式）
- 抽象语法树
- 字节码
- 有向无环图（DAG）



5.2.1 逆波兰式

- **逆波兰式**：又称**后缀式**表示法。
- 一个表达式E的后缀形式的递归定义：
 1. 如果E是一个变量或常量，则E的后缀式是E自身。
 2. 如果E是 $E_1 \text{ op } E_2$ 形式的表达式，其中op是任何二元操作符，则E的后缀式为 $E_1' E_2' \text{ op}$ ，其中 E_1' 和 E_2' 分别为 E_1 和 E_2 的后缀式。
 3. 如果E是 (E_1) 形式的表达式，则 E_1 的后缀式就是E的后缀式。

5.2.1 逆波兰式

运算对象写在前，运算符写在后 (后缀表示形式)

例： $a+b \rightarrow$

$ab+$

$(a+b)*c \rightarrow$

$ab+c*$

$a+b*c \rightarrow$

$abc*+$

$a=b*c+b*d \rightarrow$

$abc*bd*+=$



5.2.1 逆波兰式

逆波兰表示法不用括号。只要知道每个算符的目数，对于后缀式，不论从哪一端进行扫描，都能对它进行唯一分解。

优点：易于计算机处理

利用栈，将扫描到的运算对象入栈，碰到运算符：

若是双目运算符，则对栈顶的两个运算对象实施该运算并将运算结果代替这两个运算对象进栈；若是单目运算符，对栈顶元素，执行该运算，将运算结果代替该元素进栈，最后结果即栈顶元素。

练习

写出下列算式的逆波兰表示

$$a+b*(c+d/e)$$

$$a=b*(-c)+b*(-34)$$



$$abcde/+*+$$

$$a\ b\ c\ -\ * \ b\ 34\ -\ * \ +\ =$$

5.2.2 三地址代码

- 三地址代码是形如 $x = y \text{ op } z$ 的指令集合。指令 $x = y \text{ op } z$ 最多具有三个地址：两个运算分量 y 和 z ，一个结果变量 x 。
- 三地址代码可以对多运算符算术表达式和控制流语句的嵌套结构进行拆分，适用于目标代码的生成和优化。
- 三地址代码基于两个基本的概念：地址和指令。地址就是运算分量，指令就是运算符，一个地址的表现形式可以是变量名、常量或者编译器生成的临时变量。下面是几种常见的三地址指令形式：

5.2.2 三地址代 码

几种常见的三地址指令形式

指令	形式	描述
赋值指令	$x = y \text{ op } z$	op 是双目运算符或逻辑运算符, x、y、z 是地址
	$x = \text{op } y$	op 是单目运算符, y 是地址
复制指令	$x = y$	把 y 的值赋给 x
无条件转移指令	goto L	下一步要执行的指令是带有标号 L 的三地址指令
条件转移指令	if x goto L	当 x 为真时, 下一步执行带有标号 L 的指令, 否则下一步执行序列中的后一条指令
	if FALSE x goto L	当 x 为假时, 下一步执行带有标号 L 的指令, 否则下一步执行序列中的后一条指令
	if x relop y goto L	当 x 和 y 满足 relop 关系时, 下一步执行带有标号 L 的指令, 否则下一步执行序列中的后一条指令
参数传递和过程调用指令	param x_1 ... param x_n call p, n	"param x" 进行参数传递, "call p, n" 进行过程调用。"call p, n" 中的 p 表示过程, n 表示参数个数。这个 n 是必须的, 因为前面的一些 param 指令可能是 p 返回之后才执行的某个函数调用的参数
带下标的复制指令	$x = y[i]$	把距离位置 y 处 i 个内存单元的位置中存放的值赋给 x
	$x[i] = y$	把距离位置 x 处 i 个内存单元的位置中的内容设置为 y
地址及指针赋值指令	$x = \&y$	把 x 的值设置为 y 的地址
	$x = *y$	把位置 y 中存放的值赋给 x
	$*x = y$	把位置 x 中的内容设置为 y

5.2.2 三地址代码

• 1 四元式

目前最常用的中间代码形式：

(运算符，第一运算数，第二运算数，结果)

例： $a = b * c + b * d$

(1) $(*, b, c, t1)$

(2) $(*, b, d, t2)$

(3) $(+, t1, t2, t3)$

(4) $(=, t3, , a)$

也可以写成赋值语句形式：

(1) $t1 = b * c$

(2) $t2 = b * d$

(3) $t3 = t1 + t2$

(4) $a = t3$

5.2.2 三地址代码

• 2 三元式

- 编号（运算符，第一运算数，第二运算数）

如： $a = b * c + b * d$

(1) ($*$, b, c)

(2) ($*$, b, d)

(3) ($+$, (1), (2))

(4) ($=$, (3), a)

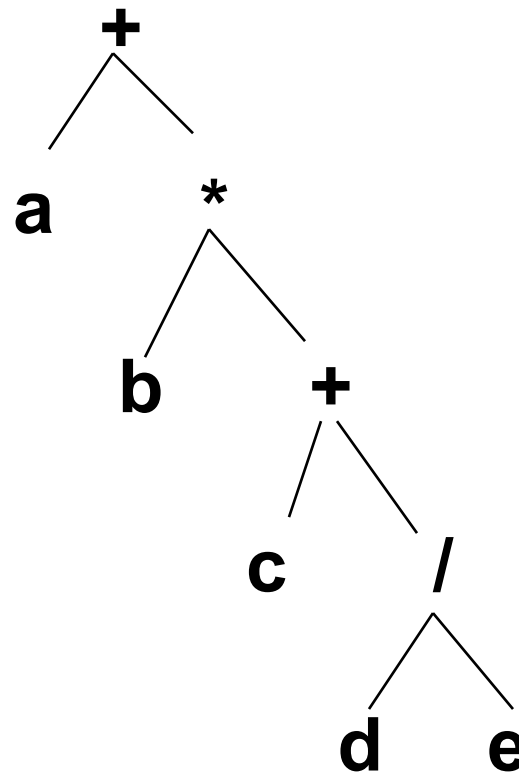
- 对于单目运算符，只有一个运算对象，另一个为空
- 注意：在三元式中的编号既代表了序号，又代表了结果的存放位置。

5.2.3 有向无环图

表达式的有向无环图

(Directed Acyclic Graph, 简称DAG) 与语法分析树类似, 一个DAG的叶子结点对应于原子运算分量, 内部结点对应于运算符。

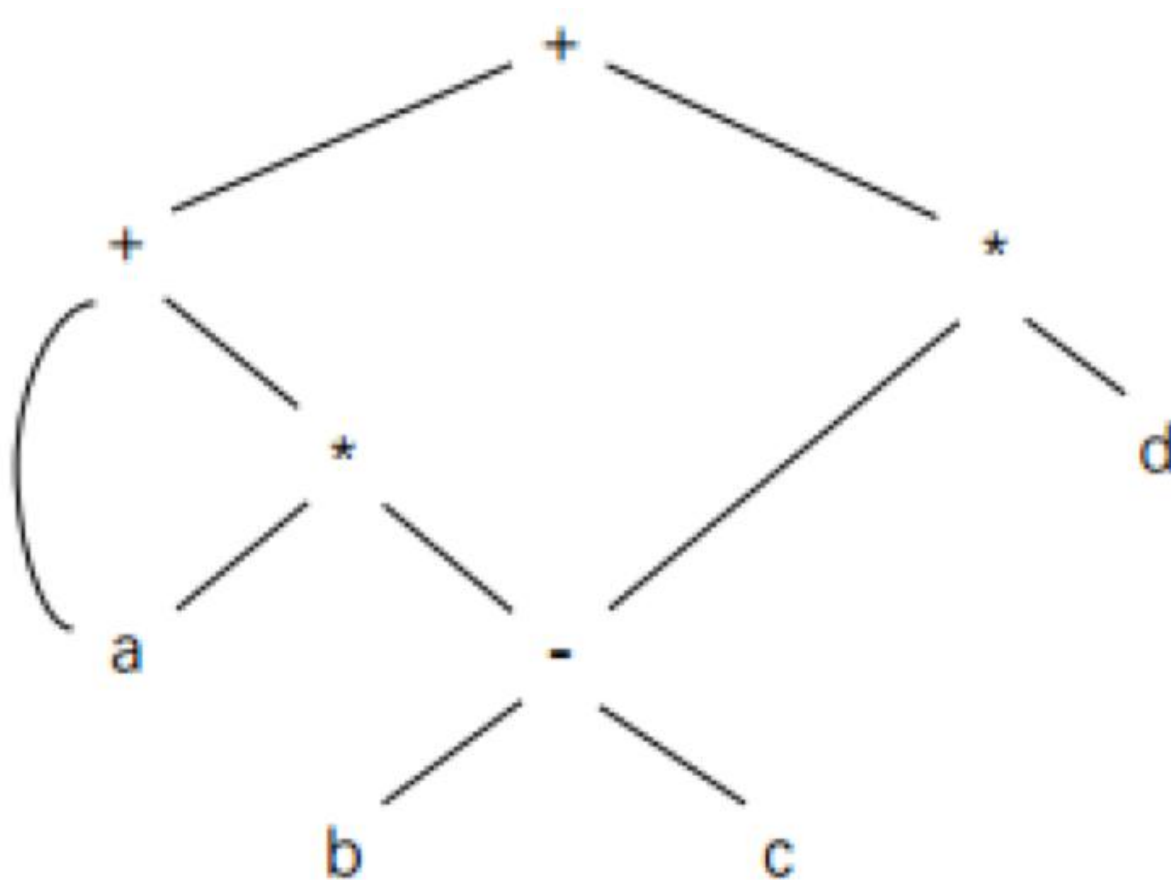
与语法分析树不同的是, 如果DAG中一个结点N表示一个公共子表达式, 那么N可能有多个父结点。



$a + b * (c + d / e)$ 的DAG

练习

表达式 $a + a * (b - c) + (b - c) * d$ 的 DAG 表示



5.2.3 有向无环图



DAG的每个结点通常作为一条记录被存放在数组中，一个记录包括的结点信息有：

结点标记：如果记录表示叶子结点，那么结点标记是变量名或者常数；如果记录表示内部结点，那么结点标记是运算符，代表进行这种运算和计算出来的值；图中各个结点可能附加一个或多个标记标识符，表示这些标识符都具有该结点所代表的值。

词法值：如果记录表示叶子结点，那么记录还包括该结点的词法值，通常是一个指向符号表的指针或者一个常量；

左右子结点：如果记录表示内部结点，那么记录还包括该结点的左右子结点。根据作用到一个名字上的标识符可以决定需要一个名字的左值还是右值。

到目前为止，已知输入的是带有语义值的
语法单位，输出是要翻译的结果的形式

需要解决的问题：
语义表示和翻译的方法

- 语义表示较流行的方法是属性文法
- 翻译方法是语法制导的翻译



5.3 属性文法和语法制导的翻译

5.3.1 属性文法

- 属性文法（也称属性翻译文法）

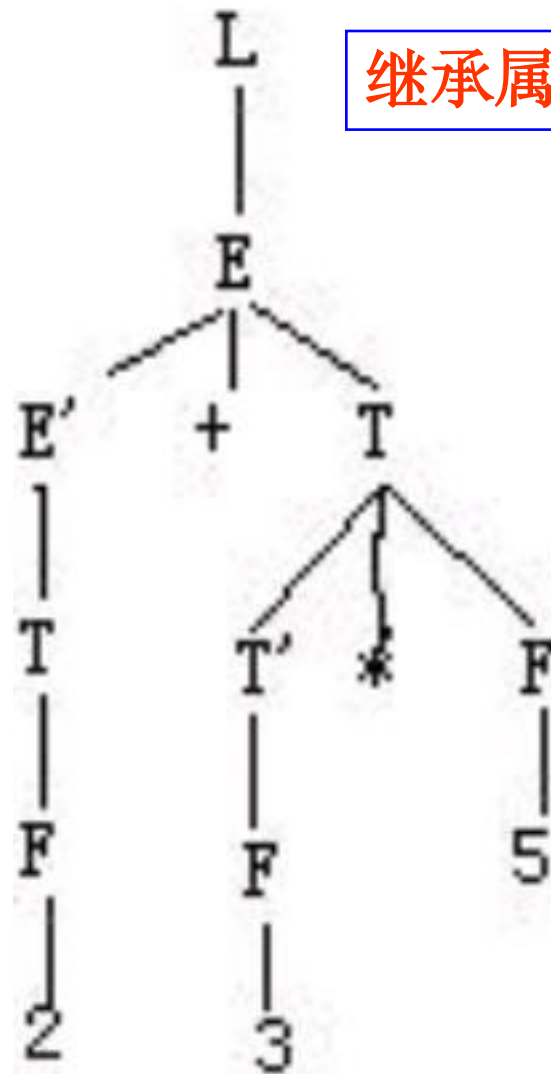
- Knuth在1968年提出
- 在上下文无关文法的基础上，为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性）。
 - 属性代表与文法符号相关信息，如类型、值、代码序列、符号表内容等
 - 属性可以进行计算和传递
 - 语义规则：对于文法的每个产生式都配备了一组属性的计算规则

5.3.1 属性文法

- 属性分两类：
 - 综合属性：用于自下而上传递信息
 - 继承属性：用于自上而下传递信息

综合属性

继承属性



5.3.1 属性文法



在一个属性文法中，对应于每个产生式 $A \rightarrow \alpha$ 都有一套与之相关联的语义规则，每条规则的形式为：

$b = f(c_1, c_2, \dots, c_k)$ 这里， f 是一个函数，

1. 如果 b 是 A 的一个属性，且 $c_1, c_2, \dots, c_k$ 是产生式右边文法符号的属性，或者是 A 的其他属性，则称 b 为 A 的综合属性。
2. b 是产生式右边某个文法符号 X 的一个属性，且 $c_1, c_2, \dots, c_k$ 是 A 或产生式右边任何文法符号的属性。则称 b 为 X 的继承属性

在这两种情况下，属性 b 依赖于属性 $c_1, c_2, \dots, c_k$ 。

5.3.1 属性文法

- 终结符只有综合属性，由词法分析器提供
- 非终结符既可有综合属性也可有继承属性，文法开始符号的所有继承属性作为属性计算前的初始值
- 对出现在产生式右边的继承属性和出现在产生式左边的综合属性都必须提供一个计算规则。属性计算规则中只能使用相应产生式中的文法符号的属性
- 出现在产生式左边的继承属性和出现在产生式右边的综合属性不由所给的产生式的属性计算规则进行计算，它们由其它产生式的属性规则计算或者由属性计算前的参数提供

5.3.2 属性的计算

- 语义规则所描述的工作可以包括属性计算、静态语义检查、符号表操作、代码生成等等。
- 例，考虑非终结符A，B和C，其中，A有一个继承属性a和一个综合属性b，B有综合属性c，C有继承属性d。产生式 $A \rightarrow BC$ 可能有规则

C. $d := B.c + 1$

A. $b := A.a + B.c$

5.3.2 属性的计算

产生式

$L \rightarrow E \#$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语 义 规 则

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$

5.3.2 属性的计算

- 综合属性的计算
 - 在语法树中，一个结点的综合属性的值由其子结点的属性值确定
 - 使用自底向上的方法在每一个结点处使用语法规则计算综合属性的值
- 仅仅使用综合属性的属性文法称S - 属性文法

5.3.2 属性的计算

产生式

$L \rightarrow E \#$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语 义 规 则

$\text{print}(E.val)$

$E.val := E_1.val + T.val$

$E.val := T.val$

$T.val := T_1.val * F.val$

$T.val := F.val$

$F.val := E.val$

$F.val := \text{digit.lexval}$

产生式 语义规则

$L \rightarrow E \#$ $\text{print}(E.\text{val})$

$E \rightarrow E_1 + T$ $E.\text{val} :=$
 $E_1.\text{val} + T.\text{val}$

$E \rightarrow T$ $E.\text{val} := T.\text{val}$

$T \rightarrow T_1 * F$ $T.\text{val} := T_1.\text{val} * F.\text{val}$

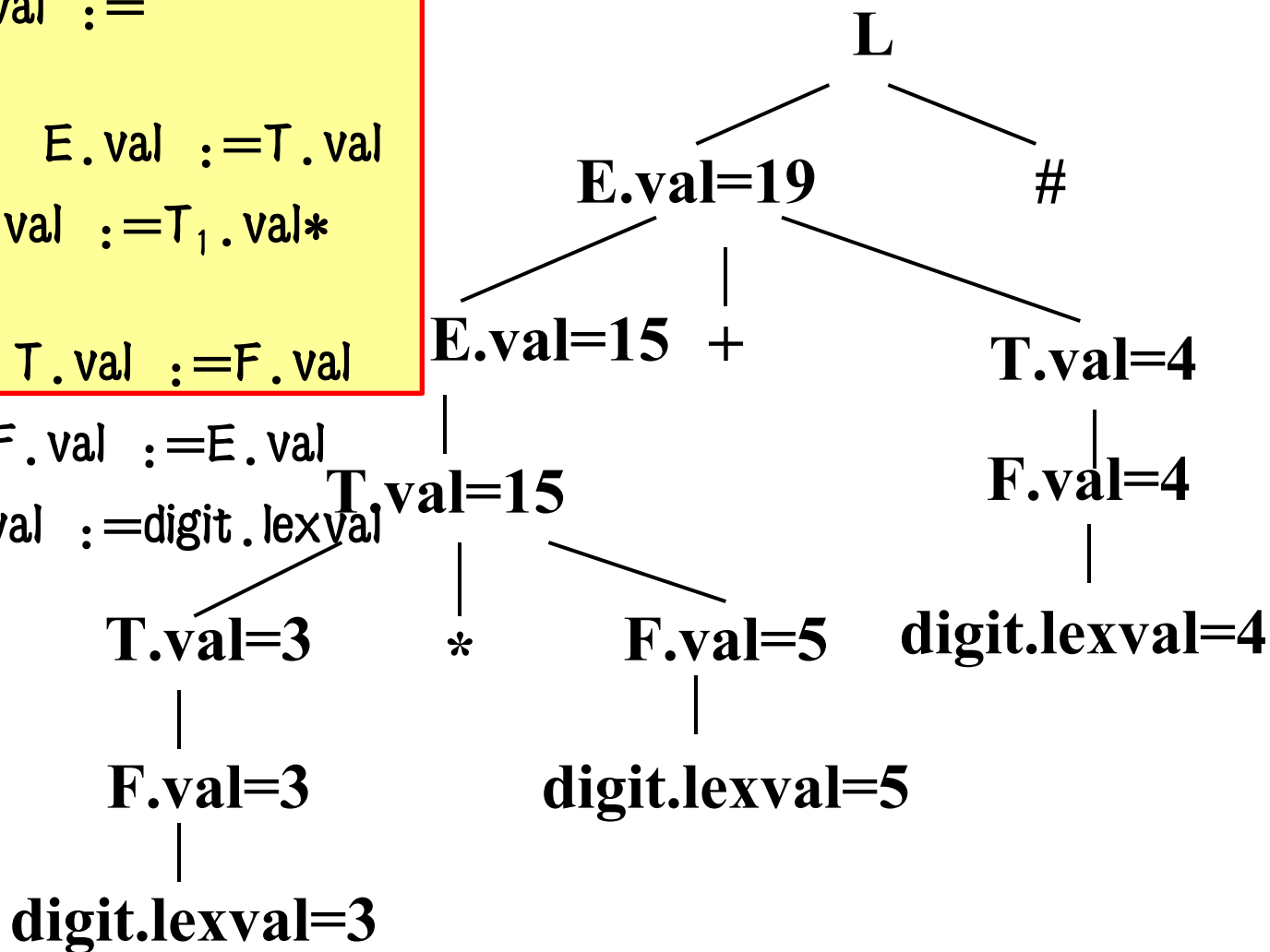
$T \rightarrow F$ $T.\text{val} := F.\text{val}$

$F \rightarrow (E)$ $F.\text{val} := E.\text{val}$

$F \rightarrow \text{digit}$ $F.\text{val} := \text{digit}.\text{lexval}$



3*5+4#的带注释的语法树



5.3.2 属性的计算

- 继承属性的计算

- 在语法树中，一个结点的继承属性由此结点的父结点和/或兄弟结点的某些属性确定
- 用继承属性来表示程序设计语言结构中的上下文依赖关系很方便



5.3.2 属性的计算

C语言中标识符的属性文法定义

产生式

语义规则

$D \rightarrow TL$ $L.in := T.type$

$T \rightarrow int$ $T.type := integer$

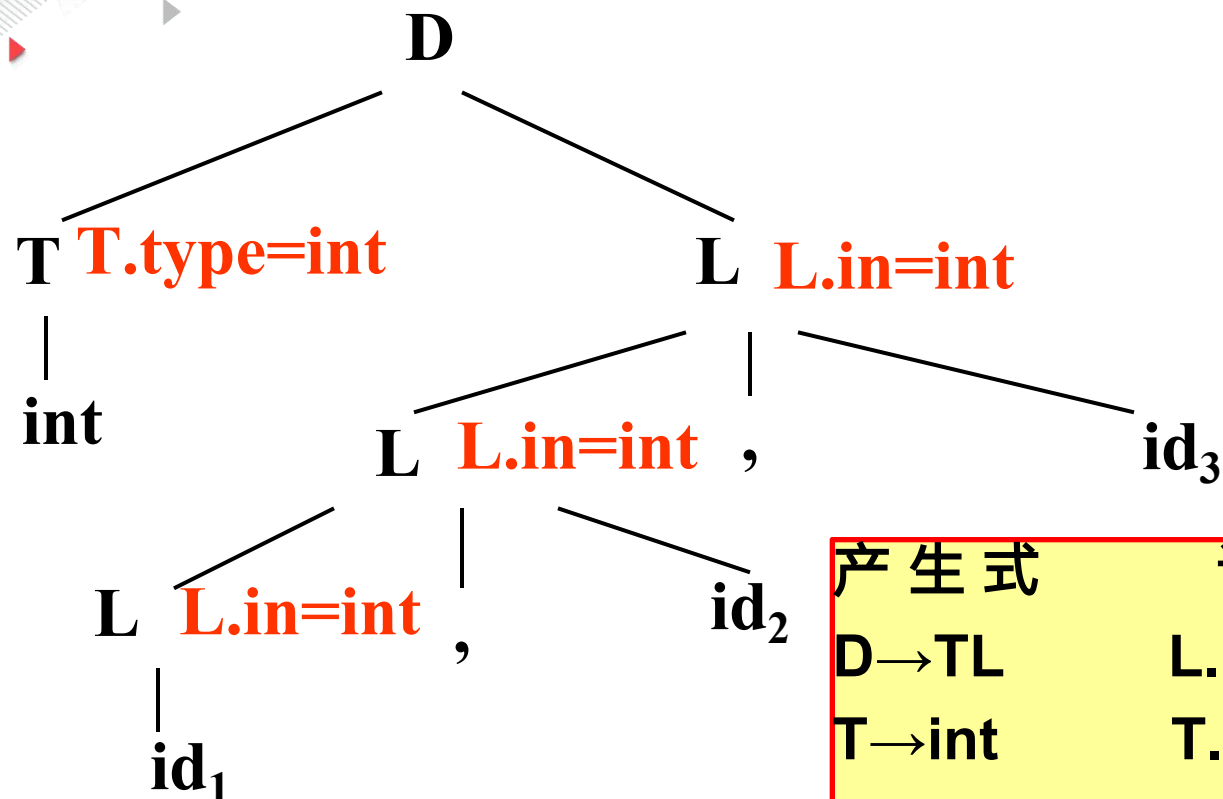
$T \rightarrow real$ $T.type := real$

$L \rightarrow L_1, id$ $L_1.in := L.in$

$addtype(id.entry, L.in)$

$L \rightarrow id$ $addtype(id.entry, L.in)$

句子 $\text{int } id_1, id_2, id_3$ 的带注释的语法树



产生式

语义规则

$D \rightarrow TL$

$L.in := T.type$

$T \rightarrow \text{int}$

$T.type := \text{int}$

$T \rightarrow \text{real}$

$T.type := \text{real}$

$L \rightarrow L_1, id$

$L_1.in := L.in$

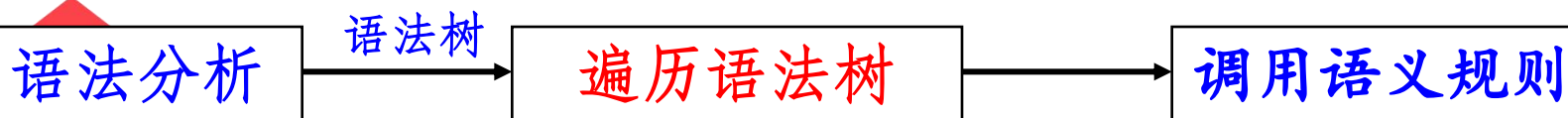
$\text{addtype}(id.entry, L.in)$

$L \rightarrow id$

$\text{addtype}(id.entry, L.in)$

5.3.3 属性的计算顺序

基于属性文法的语义处理过程实际上就是属性的计算过程，首先对单词符号串进行语法分析，构造语法树，然后根据需要遍历语法树，在语法树的各结点处按照语义规则进行属性计算。



小结

- 语义规则描述的工作：

- 属性计算、静态语义检查、符号表的操作、代码生成等，通常写成过程和函数调用，称为**语义子程序**。



- 语义翻译常用的方法：

- **语法制导翻译**：定义翻译所必须的语义属性和语义规则，一般不涉及计算顺序。

5.3.4 语法制导翻译的实现方法

- 语法制导翻译 (Syntax-Directed Translations) :
 - 一个句子的语义翻译过程与语法分析过程同时进行。
 - 在文法中，文法符号有明确的意义，文法符号之间有确定的语义关系。属性描述语义信息，语义规则描述属性间的关系，将语义规则与语法规则相结合，在语法分析的过程中计算语义属性值。

5.3.4 语法制导翻译的实现方法

- 对应每一个产生式编制一个语义子程序，在进行语法分析的过程中，当一个产生式获得匹配时，就调用相应的语义子程序实现语义检查与翻译。即在自下而上语法分析的过程中，每一次归约时就调用相应的语义子程序。
- 在产生式的右部的适当位置，插入相应的语义动作，按照分析的进程，执行遇到的语义动作。

5.3.4 语法制导翻译的实现方法

一个语义子程序描述了一个产生式对应的翻译工作。

- 主要有：改变编译程序某些变量的值、查填各种符号表、发现并报告源程序错误、产生中间代码。
- 在描述语义动作时需要为每个文法符号赋以各种不同的语义值，如类型、地址、代码值等。
- 如果一个产生式中一个符号多次出现，就用上角标来区分，如： $E = E^{(1)} + E^{(2)}$

5.3.4 语法制导翻译的实现方法

❖ 语义子程序的一般形式

❖ 语义子程序写在该产生式后面的花括号内：

❖ (1) $X \rightarrow \dots$ { 语义子程序1 }

❖ (2) $Y \rightarrow \dots$ { 语义子程序2 }

❖ (3) $A \rightarrow XY$ { 语义子程序3 }

⌘ 例：台式计算器程序的语义子程序描述：

产生式	语义子程序
⌘ (0) $S' \rightarrow E$	{PRINT E. VAL}
⌘ (1) $E \rightarrow E^{(1)} + E^{(2)}$	{E. VAL = E ⁽¹⁾ . VAL + E ⁽²⁾ . VAL}
⌘ (2) $E \rightarrow E^{(1)} * E^{(2)}$	{E. VAL = E ⁽¹⁾ . VAL * E ⁽²⁾ . VAL}
⌘ (3) $E \rightarrow (E^{(1)})$	{E. VAL = E ⁽¹⁾ . VAL}
⌘ (4) $E \rightarrow i$	{E. VAL = LEXVAL}

5.3.4 语法制导翻译的实现方

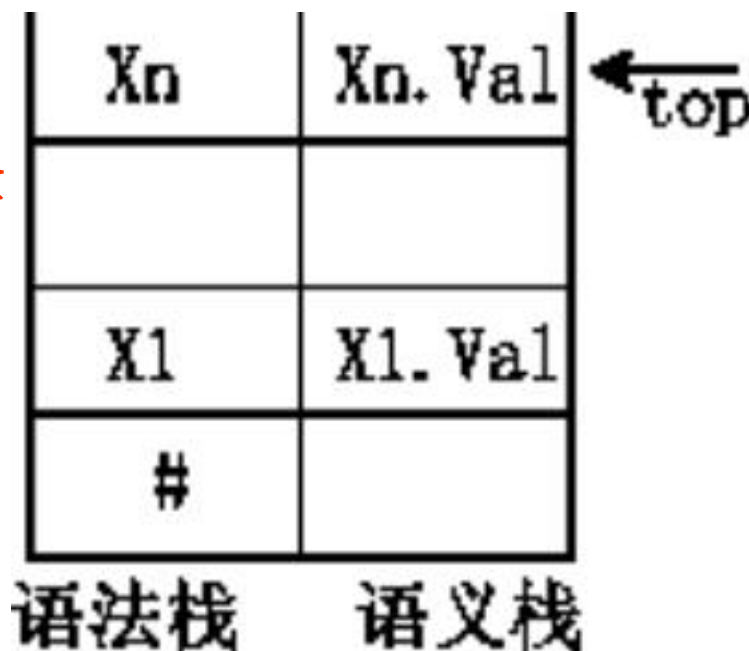


法 • 常见的语法制导翻译类型:

- 语法分析采用自下而上方法时，使用与语法分析栈同步操作的**语义栈**进行语法制导翻译。
- 语法分析采用递归下降分析方法时，利用**隐含堆栈**存储各递归子程序中的局部变量所表示的语义信息
- 语法分析采用LL(1)分析法时，利用翻译文法进行语法制导的翻译。翻译文法是在描述语言的**文法中加入语义动作**的符号。
- S-属性文法翻译。属性文法也是一种翻译文法，它的文法符号和动作符号都带有语义属性和同一产生式中各属性间的运算规则。

5.3.4 语法制导翻译的实现方法

S-属性文法的自下而上翻译



- 使用上图形式的栈实现——增加语义栈
- 用 $X.Val$ 表示文法符号 X 的语义信息。
- 语义信息与文法符号同时出栈和入栈
- 下面以一个例子来说明自底向上的翻译方法

5.3.4 语法制导翻译的实现方法

例1：下面是一个算术表达式文法，每个产生式右边是它的语义动作，对输入串 $2 * 3 + 2$ 的规范归约的分析过程如下：

产生式	语义子程序
$E \rightarrow E^{(1)} + E^{(2)}$	$\{E \cdot V \text{ AL} = E^{(1)} \cdot V \text{ AL} + {}^i E^{(2)} \cdot V \text{ AL}\}$
$E \rightarrow E^{(1)} * E^{(2)}$	$\{E \cdot V \text{ AL} = E^{(1)} \cdot V \text{ AL} * {}^i E^{(2)} \cdot V \text{ AL}\}$
$E \rightarrow 0 1 2 \cdots 9$	$\{E \cdot V \text{ AL} = d d = 0, 1, \dots, 9\}$

5.3.4 语法制导翻译的实现方法

输入串
2 * 3 + 2 #
↑

—	#

(1) 初态
语法树为空

* 3 + 2 #
↑

	2
	#

(2) 移进 2
语法树为空

* 3 + 2 #
↑

E • VAL = 2	E
—	#

(3) 归约 2
E(2)
|
2

3 + 2 #
↑

* i	*
E • VAL = 2	E
—	#

(4) 移进 *
E(2)
|
2

5.3.4 语法制导翻译的实现方

输入串 $+2\#$
↑

	3
$\star i$	$\star$
$E \cdot VAL = 2$	E
—	#

(5) 移进 3
E(2)

语法树
|
2 $\star$ 3

$+2\#$
↑

$E \cdot VAL = 3$	E
$\star i$	$\star$
$E \cdot VAL = 2$	E
—	#

(6) 归约 3
E(2) E(3)

| $\star$ |
2 3

$+2\#$
↑

$E \cdot VAL = 6$	E
—	#

(7) 归约 $E \star E$

E(6)
/ | \
E(2) $\star$ E(3)
| | |
2 $\star$ 3

$2\#$
↑

$+i$	$+$
$E \cdot VAL = 6$	E
—	#

(8) 移进 +
E(6)

/ | \
E(2) $+$ E(3)
| | |
2 $+$ 3 $+$

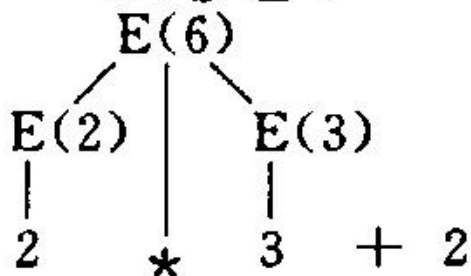
5.3.4 语法制导翻译的实现方

法

↑

	2
+ ⁱ	+
E·VAL=6	E
—	#

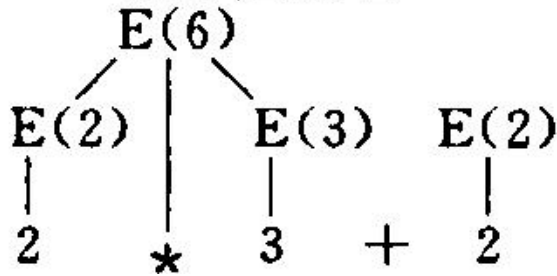
(9) 移进 2



↑

E·VAL=2	E
+ ⁱ	+
E·VAL=6	E
—	#

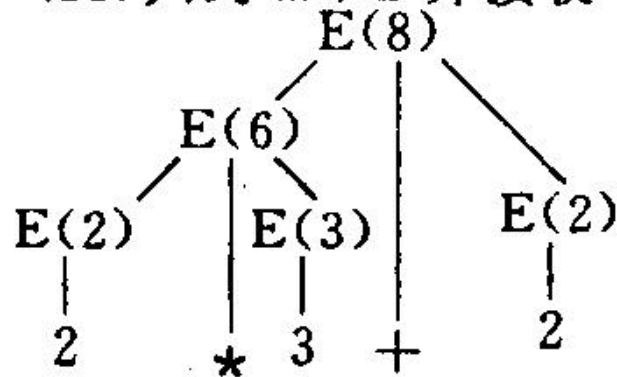
(10) 归约 2



↑

E·VAL=8	E
—	#

(11) 归约 E+E 并接收



 河南师范大学
HENAN NORMAL UNIVERSITY

$$\begin{aligned} (1) \quad & E \rightarrow E + T \mid T \\ (2) \quad & T \rightarrow T * F \mid F \\ (3) \quad & F \rightarrow (E) \mid i \end{aligned}$$
[illegible]



序号	状态栈	符号栈	语义栈	归约产生式	输入串
1	0	#	-		3*5+4#
2	05	#3	--		*5+4#
3	03	#F	-3	$F \rightarrow i$	*5+4#
4	02	#T	-3	$T \rightarrow F$	*5+4#
5	027	#T*	-3-		5+4#
6	0275	#T*5	-3--		+4#
7	0271 <u>0</u>	#T*F	-3-5	$F \rightarrow i$	+4#
8	02	#T	-15	$T \rightarrow T*F$	+4#
9	01	#E	-15	$E \rightarrow T$	+4#
10	016	#E+	-15-		4#
11	0165	#E+4	-15--		#
12	0163	#E+F	-15-4	$F \rightarrow i$	#
13	0169	#E+T	-15-4	$T \rightarrow F$	#
14	01	#E	-19	$E \rightarrow E+T$	#
15				接受	

5.3.4 语法制导翻译的实现方法

- 在刚才的翻译过程中有如下特点：
 - 句柄归约在先，语义动作在后。
 - 归约时，栈内句柄各符号的语义信息与该句柄同时出栈，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量同时入栈。
 - 为了在某处调用语义动作，就必须先归约，因此，有时需要改写文法。



小结

语法单位 (输入)

中间代码 (要翻译的结果)

属性文法 (语义规则的表示方法)

语法制导翻译技术 (翻译的方法)

5.4 常见语句的语法制导的翻译



- 高级语言的语句分类：
 - 说明语句——定义各种名字的属性
 - 可执行语句——用于完成指定的功能，涉及到指令代码
- 语义翻译也分两类：
 - 翻译说明语句时，将所定义的名字的各种属性登记到符号表中
 - 翻译可执行语句时，首先应根据各源语句的语法结构和语义设计出它的目标代码结构，找出源与目标的对应关系，给出对信息(数据表示)描述和从源到目标的变换算法。语义子程序则根据变换方法进行翻译。

语法制导翻译技术翻译过程中的特点：

- 句柄归约在先，语义动作在后。
- 归约时，栈内句柄各符号的语义信息与该句柄**同时出栈**，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量**同时入栈**。
- 为了在某处调用语义动作，就必须先归约，因此，有时需要**改写文法**。



5.4.1 声明语句的语义处理

声明语句的作用：

- 就是说明类型等属性信息，在翻译时主要是填符号表

声明语句分类：

- 变量声明语句
- 常量声明语句

一 语义变量和语义函数

- 涉及到的数据结构
 - 符号表 中间代码表 临时变量区
- 涉及到的语义变量和函数
 - **X.PLACE**: 表示与X对应的变量在符号表中的位置。
 - **ENTRY(id)**: 函数, 查找变量id的入口地址; 检查id.name是否在符号表中, 如在则返回一指向该登陆项的指针, 否则返回NULL

二 常量声明语句

1. 常量说明语句的文法描述

- (1) $\langle \text{CONSTDCL} \rangle \rightarrow \text{const} \langle \text{CONSTDEF} \rangle$
- (2) $\rightarrow \varepsilon$
- (3) $\langle \text{CONSTDEF} \rangle \rightarrow \langle \text{CONSTDEF} \rangle; \text{id} = \text{INT}$
- (4) $\rightarrow \langle \text{CONSTDEF} \rangle; \text{id} = \text{STR}$
- (5) $\langle \text{CONSTDEF} \rangle \rightarrow \text{id} = \text{INT}$
- (6) $\rightarrow \text{id} = \text{STR}$

例: **const A = 123; K = 'ABC'**

2. 常量说明语句的翻译

策略: 先读完一个说明语句再翻译。

先翻译后面的产生式, 再翻译前面的产生式,
在归约时, 执行语义动作, 将常量的值、类型、
种属填入符号表。

3. 翻译的语义动作

产生式	语义处理
(5) <CONSTDEF> → id=INT	{ entry(id).val= entry(INT).val; entry(id).type=integer; entry(id).kind=数值常数; }
(6) <CONSTDEF> → id=STR	{ entry(id).val= entry(STR).val; entry(id).type=char; entry(id).kind=字符常数; }

将常数的类型填入符号表的Type栏
将常数的种属填入符号表的Kind栏

3, 4产生式的翻译与5, 6产生式的翻译相同
1, 2产生式没有语义动作

const A=123; K= 'ABC'

Name		Type	Kind	Val	Addr
A	1	integer	数值常数	123	
K	1	char	字符常数	ABC	

三 变量声明语句

1. 变量说明语句的文法描述

- (1) $\langle \text{VARDCL} \rangle \rightarrow \text{var } \langle \text{IDS} \rangle \mid \varepsilon$
- (2) $\langle \text{IDS} \rangle \rightarrow \text{id}, \langle \text{IDS} \rangle^{(1)}$
- (3) $\langle \text{IDS} \rangle \rightarrow \text{id:integer}$
- (4) $\langle \text{IDS} \rangle \rightarrow \text{id:char}$
- (5) $\langle \text{IDS} \rangle \rightarrow \text{id: bool}$
- (6) $\langle \text{IDS} \rangle \rightarrow \text{id: real}$

2. 变量说明语句的翻译

var exam, b, c: integer;

策略：先翻译第 3，4 产生式，再翻译 2，1 产生式，以便记录 $\langle \text{IDS} \rangle$ 的类型，即在写程序时，**读完**一个说明语句，开始归约，再开始翻译，变量的类型朝前传递。

3. 翻译的语义动作

entry(id) 表示变量名id
在符号表中的入口

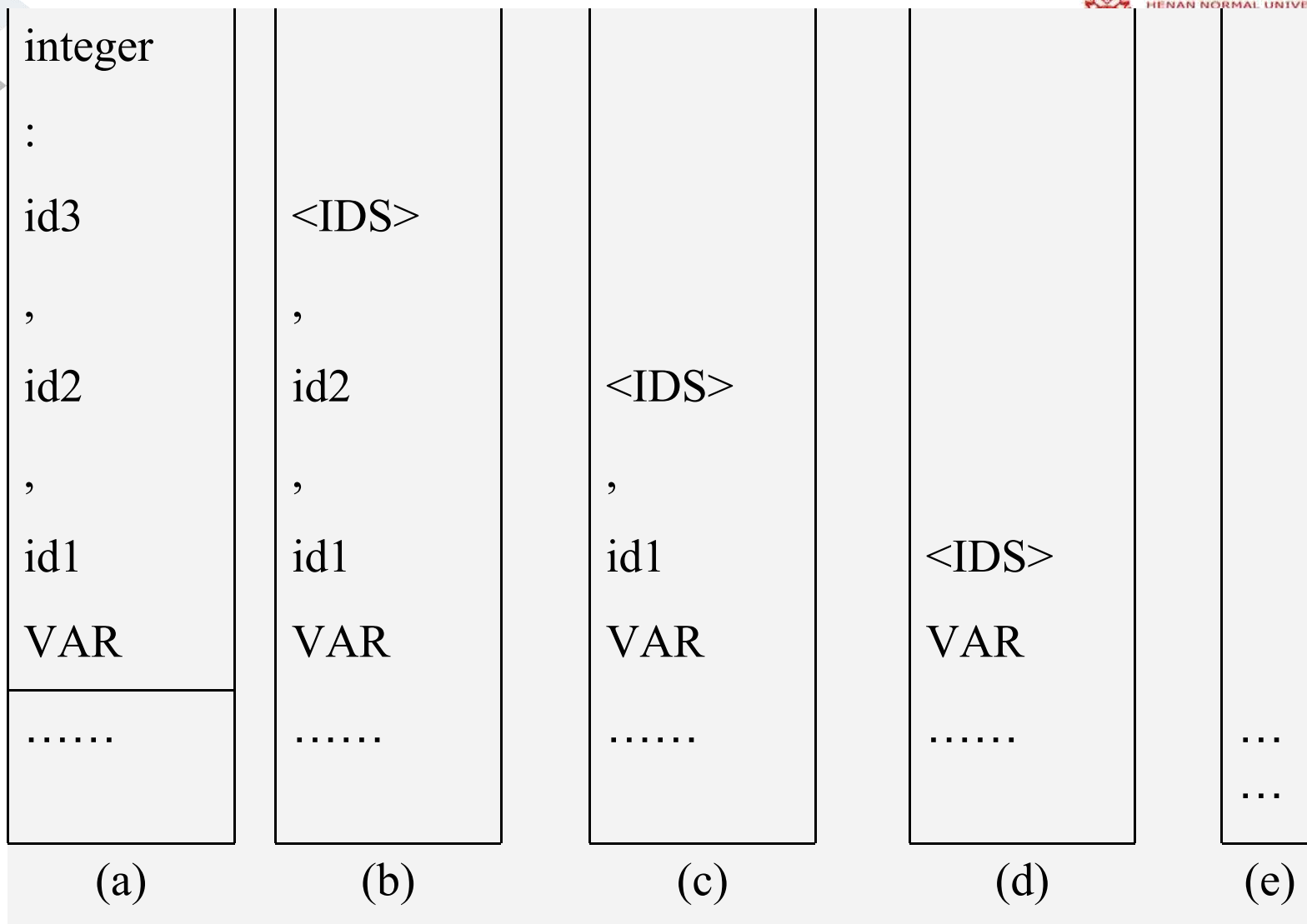
编号	产生式	语义动作
(4)	$\langle \text{IDS} \rangle \rightarrow \text{id}:\text{char}$	{ fill(entry(id),char); $\langle \text{IDS} \rangle \cdot \text{type} = \text{char}$ }
(3)	$\langle \text{IDS} \rangle \rightarrow \text{id}:\text{integer}$	{ fill(entry(id),integer); $\langle \text{IDS} \rangle \cdot \text{type} = \text{integer}$ }
(2)	$\langle \text{IDS} \rangle \rightarrow \text{id}, \langle \text{IDS} \rangle^{(1)}$	{ fill(entry(id), $\langle \text{IDS} \rangle^{(1)} \cdot \text{type}$); $\langle \text{IDS} \rangle \cdot \text{type} = \langle \text{IDS} \rangle^{(1)} \cdot \text{type}$ }
(1)	$\langle \text{VARDCL} \rangle \rightarrow \text{var } \langle \text{IDS} \rangle \epsilon$	{ nop }

var exam, b, c: integer;

符号表:

Name		TYPE	KIND	VAL	ADDR
exam	4	integer			
b	1	integer			

VAR id1,id2,id3:integer;的归约过程



5.4.2 执行语句

- 定义的一些语义变量和过程
 - **GENCODE(OP,ARG1,ARG2,RESULT)**：语义过程，产生一个四元式，并填入四元式表，编号自动增1。
 - **NEWT**：函数，返回一个临时变量序号。在翻译可执行语句的过程中，可能需要使用临时变量，假定用NEWT过程来申请临时变量Ti，每申请一次，i增1。
 - **E.PLACE**：与非终结符E相联系的语义变量，可能是某变量的入口地址，或者为临时变量序号。
 - **NXQ**：即将要生成的四元式的编号。

一 简单赋值语句的翻译

1. 简单赋值语句的文法描述

例如: $x = 2 + 3 * 2;$

$$S \rightarrow id = E$$
$$E \rightarrow E_1 + T \mid T$$
$$T \rightarrow T_1 * F \mid F$$
$$F \rightarrow -F \mid id \mid (E)$$

2. 简单赋值语句的代码结构

$id = aexpr;$

aexpr的代码

$id = aexpr.place$

3. 简单赋值语句的翻译

编号	产生式	语义规则
(1)	$S \rightarrow id = E$	{ gencode(=, E.PLACE, _, entry(id)) }
(2)	$E \rightarrow E_1 + T$	{ E.PLACE = newtemp(); gencode(+i, E ₁ .PLACE, T.PLACE, E.PLACE) }
(3)	$E \rightarrow T$	{ E.PLACE = newtemp(); E.PLACE = T.PLACE }
(4)	$T \rightarrow T_1 * F$	{ T.PLACE = newtemp(); gencode(*i, T ₁ .PLACE, F.PLACE, T.PLACE) }
(5)	$T \rightarrow F$	{ T.PLACE = newtemp(); T.PLACE = F.PLACE }
(6)	$F \rightarrow -F_1$	{ F.PLACE = newtemp(); gencode(@i, F.PLACE, _, F ₁ .PLACE) }
(7)	$F \rightarrow i$	{ F.PLACE = newtemp(); F.PLACE = ENTRYR(i) }
(8)	$F \rightarrow (E)$	{ F.PLACE = newtemp(); F.PLACE = E.PLACE }

例：赋值句 $A=B+C*(-D)$ 的自底向上分析

设：翻译此赋值句之前四元式的最大编号为K

$D) \#$
↑

$F_D \cdot \text{PLACE}$	F_D
@ ¹	—
(	(
* ¹	*
$T_c \cdot \text{PLACE}$	T_c
+ ¹	+
$E_B \cdot \text{PLACE}$	E_B
=	=
ENTRY(i_A)	i_A
—	#

前期全部移进

$) \#$
↑

$F'_D \cdot \text{PLACE}$	F'_D
(	(
* ¹	*
$T_c \cdot \text{PLACE}$	T_c
+ ¹	+
$E_B \cdot \text{PLACE}$	E_B
=	=
ENTRY(i_A)	i_A
—	#

归约 $-F_D$ 为 F'_D
产生四元式 K+1:

$(@^1, F_D \cdot \text{PLACE}, -, F'_D \cdot \text{PLACE})$

$\#$
↑

$F'_D \cdot \text{PLACE}$	F'_D
(	(
* ¹	*
$T_c \cdot \text{PLACE}$	T_c
+ ¹	+
$E_B \cdot \text{PLACE}$	E_B
=	=
ENTRY(i_A)	i_A
—	#

移进)

$\#$
↑

$F'_D \cdot \text{PLACE}$	F'_D
* ¹	*
$T_c \cdot \text{PLACE}$	T_c
+ ¹	+
$E_B \cdot \text{PLACE}$	E_B
=	=
ENTRY(i_A)	i_A
—	#

归约(F'_D)为 F'_D

↑

$T' \cdot \text{PLACE}$	T'
$+^1$	$+$
$E_B \cdot \text{PLACE}$	E_B
$=$	$=$
$\text{ENTRY}(i_A)$	i_A
$-$	$\#$

归约 $T_c *^i F_D^*$ 为 T'
产生四元式 $K+2$

↑

$E \cdot \text{PLACE}$	E
$=$	$=$
$\text{ENTRY}(i_A)$	i_A
$-$	$\#$

归约 $E_B +^i T'$ 为 E
产生四元式 $K+3$

↑

$-$	

归约 $i_A = E$
产生四元式 $K+4$

$K+2: (*^i, T_c \cdot \text{PLACE}, F_D^* \cdot \text{PLACE}, T' \cdot \text{PLACE})$

$K+3: (+^1, E_B \cdot \text{PLACE}, T' \cdot \text{PLACE}, E \cdot \text{PLACE})$

$K+4: (: =, E \cdot \text{PLACE}, -, \text{ENTRY}(i_A))$



因此，四元式表中增加了4条四元式：

K	... (翻译此赋值语句之前的四元式)
$K+1$	$((@^i, F_D \cdot \text{PLACE}, _, F_D' \cdot \text{PLACE}))$
$K+2$	$((*^i, T_c \cdot \text{PLACE}, F_D'' \cdot \text{PLACE}, T' \cdot \text{PLACE}))$
$K+3$	$((+^i, E_B \cdot \text{PLACE}, T' \cdot \text{PLACE}, E \cdot \text{PLACE}))$
$K+4$	$((=, E \cdot \text{PLACE}, _, \text{ENTRY}(i_A)))$

回顾

- 翻译说明语句时，将所定义的名字的各种属性登记到符号表中
- 翻译可执行语句时，首先应根据各源语句的语法结构和语义设计出它的**目标代码结构**，找出源与目标的对应关系，给出对信息(数据表示)描述和从源到目标的**变换算法**。语义子程序则根据变换方法进行翻译。
- 简单赋值语句的翻译

aexpr的代码

id=aexpr.place

五 布尔表达式的翻译

1. 布尔表达式的作用

- 用作控制语句中的条件表达式
- 用在逻辑赋值语句中

2. 布尔表达式的形式

- (1)单个布尔量;(2) 布尔量的运算not, and,or, 优先级比关系运算低;(3)关系运算: $E1 \text{ rop } E2$, $E1E2$ 是算术表达式, rop为关系运算符: $>, <, >=, <=, =$

3 . 布尔表达式的文法描述

(1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$

(2) $\langle BE \rangle \rightarrow \langle BT \rangle$

(3) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$

(4) $\langle BT \rangle \rightarrow \langle BF \rangle$

(5) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$

(6) $\langle BF \rangle \rightarrow (\langle BE \rangle)$

(7) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$

(8) $\langle BF \rangle \rightarrow i \text{ rop } i$

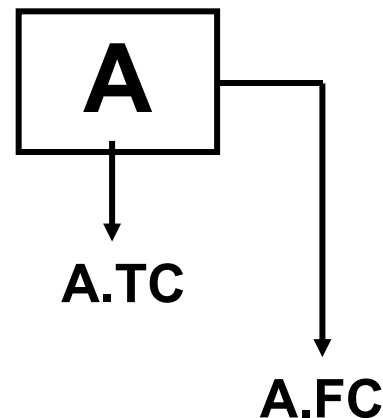
(9) $\langle BF \rangle \rightarrow i$

例如: **not a and (y>z);**

4. 布尔量的代码结构（作为控制语句的条件式）



- 每个布尔量(布尔变量或布尔常量)的目标结构有两个出口，真出口指向为真时应跳转的位置；假出口指向为假时应跳转的位置。



- 布尔量翻译为两条四元式：

- (jnz, A, _, P): 真出口，当A为真时跳转到四元式P
- (j, _, _, q): 假出口，无条件跳转到四元式q

- 关系表达式也翻译为两条四元式：

- (jrop, i1, i2, P): 真出口，当i1 rop i2为真时转四元式P
- (j, _, _, q): 假出口，无条件跳转到四元式q

在翻译布尔表达式时，后面的语句未翻译，P, q如何知道？

解决方法是：回填技术

- 已知就直接填入；不知时先填0，等知道后再返填
- 若多个因子的转移去向相同，但又不知道具体位置，应该用链将这些未知且出口相同的
- 布尔表达式： $A \text{ and } B$

当扫描到and时，对A进行归约，产生两个四元式1, 2，其中真出口已知，为3

(1) (jnz, A, —, 3)

假出口未知，填入0

(2) (j, —, —, 0)

(3) (jnz, B, —, 5)

/* B的四元式 */

(4) (j, —, —, 2)

(5) (j>, C, D, 0)

(6) (j, —, —, 4)

当扫描到B后的and时，

假出口仍未知，但对C>D时C>D生两的假出口相同，则此时链接起来，填4，真0

生成了真、假出口两个链，两个0就是待填部分。等到以后翻译到后面再填入。

- 
- 将P1, P2为链首两个四元式链合并在一起, 可以用下述过程, 返回合并后的四元式链首:



```
merge(P1, P2)
{
  if P2 == 0) return (P1);
  else {
    P := P2;
    while (四元式P的第4分量内容不为0) do
      P = 四元式P的第4分量内容;
    把P1填进四元式P的第4分量;
    return (P2);
  }
}
```

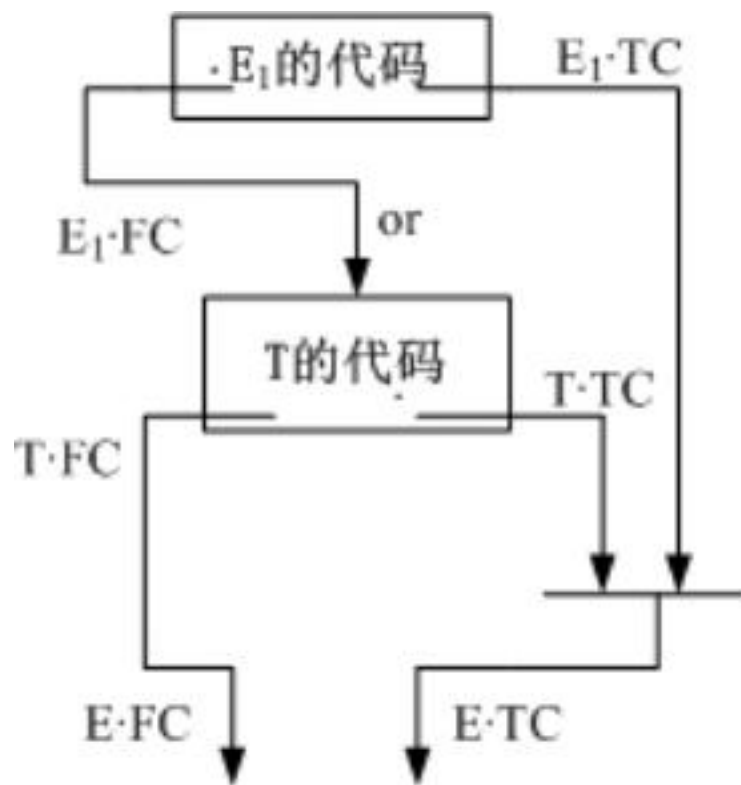

- 假出口链上的四元式应转向相同的位置，所以一旦知道转向的真实位置，就应返填，返填是将已知位置填入链上的所有四元式的第四个分量。
- 用backpatch，把已知位置t填入以P为链首的四元式中：

```
BACKPATCH(P, t)
{
    Q=P;
    while (Q!=0) do {
        m = 四元式Q的第4分量内容;
        把t填进四元式Q的第4分量;
        Q = m;
    }
}
```

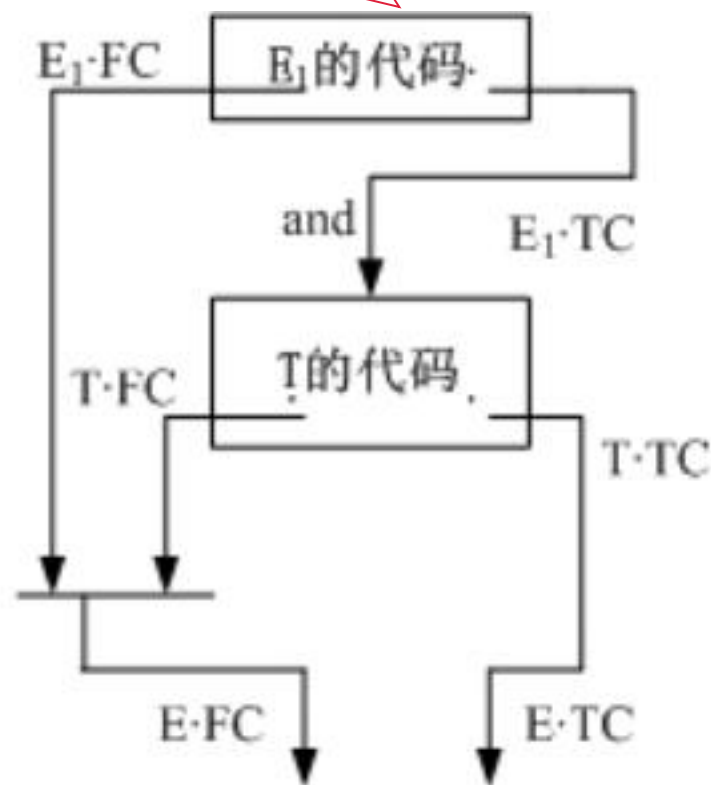
5. 布尔表达式的翻译

- 布尔表达式是带有not, and, or 的表达式
 - 第一种翻译方法可以象算术表达式一样翻译，先计算每个因子的值，再计算整个表达式的值。
 - 第二种翻译方法采取优化措施进行翻译。
-
- **E1 or T 的优化**：只要E为真，后面的表达式就不必计算，只有当E为假时才读取T，目标结构如下：
 - **E1 and T 的优化**：只要E为假，后面的表达式就不必计算，只有当E为真时才读取T，目标结构如下：

如果是翻译not E1, 则E的真假出口正好与E1相反



(a) E_1 or T的目标结构



(b) E_1 and T的目标结构



翻译布尔表达式时，当读到not, and, or时，要进行归约，因此应对文法进行改造

原因：

- 句柄归约在先，语义动作在后。
- 归约时，栈内句柄各符号的语义信息与该句柄同时出栈，整个句柄所表示的语义信息则赋给相应产生式左部非终结符的语义变量，并让该非终结符及其语义变量同时入栈。
- 为了在某处调用语义动作，就必须先归约，因此，有时需要改写文法。

翻译布尔表达式时，当读到not, and, or时，要进行归约，因此应对文法进行改造，改造前后的文法为：

- | | |
|---|---|
| (1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$ | (6) $\langle BF \rangle \rightarrow (\langle BE \rangle)$ |
| (2) $\langle BE \rangle \rightarrow \langle BT \rangle$ | (7) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$ |
| (3) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$ | (8) $\langle BF \rangle \rightarrow i \text{ rop } i$ |
| (4) $\langle BT \rangle \rightarrow \langle BF \rangle$ | (9) $\langle BF \rangle \rightarrow i$ |
| (5) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$ | |

改造前的文法

- | | |
|---|---|
| (1) $\langle BE \rangle \rightarrow \langle BE \rangle \text{ or } \langle BT \rangle$ | (7) $\langle BF \rangle \rightarrow (\langle BE \rangle)$ |
| (2) $\langle BE \rangle \text{ or } \rightarrow \langle BE \rangle \text{ or }$ | (8) $\langle BF \rangle \rightarrow \text{not } \langle BF \rangle$ |
| (3) $\langle BE \rangle \rightarrow \langle BT \rangle$ | (9) $\langle BF \rangle \rightarrow \langle AE \rangle \text{ rop } \langle AE \rangle$ |
| (4) $\langle BT \rangle \rightarrow \langle BT \rangle \text{ and } \langle BF \rangle$ | (10) $\langle BF \rangle \rightarrow i \text{ rop } i$ |
| (5) $\langle BT \rangle \text{ and } \rightarrow \langle BT \rangle \text{ and }$ | (11) $\langle BF \rangle \rightarrow i$ |
| (6) $\langle BT \rangle \rightarrow \langle BF \rangle$ | |

改造后的文法

总结前面的分析，各产生式的翻译为

NXQ表示当前产生的四元式的编号

编号	产生式	翻译为四元式
11)	$\langle BF \rangle \rightarrow i$	<pre>{ <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jnz, entry(i), _, 0); gencode(j, _, _, 0) }</pre>
10)	$\langle BF \rangle \rightarrow i^{(1)} \text{ rop } i^{(2)}$ /*rop 为<, <=, >=, ==, < 或>*/	<pre>{ <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jrop,i^{(1)}.PLACE,i^{(2)}.PLACE,0) gencode(j, _, _, 0) }</pre>
	$\langle BF \rangle \rightarrow \langle AE \rangle^{(1)} \text{ rop } \langle AE \rangle^{(2)}$ /*<AE>是算术表达式*/	<pre>{ <BF>.TC=NXQ; <BF>.FC=NXQ+1; gencode(jrop,<AE>^{(1)}.PLACE,<AE>^{(2)}.PLACE,0) gencode(j, _, _, 0) }</pre>
		<pre>{ <BF>.TC=<BF>^{(1)}.FC; <BF>.FC=<BF>^{(1)}.TC }</pre>

单个的布尔量

关系运算

带算术运算的关系运算

Not逻辑运算



产生式6: $\langle BT \rangle \rightarrow \langle BF \rangle$, 3: $\langle BE \rangle \rightarrow \langle BT \rangle$ 的翻译与7相似, 都是将右边的真假出口直接赋值到左边

编号	产生式	语义规则
(7)	$\langle BF \rangle \rightarrow (\langle BE \rangle)$	$\{ \langle BF \rangle.TC = \langle BE \rangle.TC;$ $\langle BF \rangle.FC = \langle BE \rangle.FC \}$
(5)	$\langle BT \rangle_{and} \rightarrow \langle BT \rangle \text{ and }$	$\{ \text{backpatch}(\langle BT \rangle.TC, NXQ);$ $\langle BT \rangle_{and}.FC = \langle BT \rangle.FC;$
(4)	$\langle BT \rangle \rightarrow \langle BT \rangle_{and} \langle BF \rangle$	$\{ \langle BT \rangle.TC = \langle BT \rangle_{and}.TC;$ $\langle BT \rangle.FC = \text{merge}(\langle BT \rangle_{and}.FC, \langle BF \rangle.FC) \}$
(2)	$\langle BE \rangle_{or} \rightarrow \langle BE \rangle \text{ or }$	$\{ \text{backpatch}(\langle BE \rangle.FC, NXQ);$ $\langle BE \rangle_{or}.TC = \langle BE \rangle.TC;$
(1)	$\langle BE \rangle \rightarrow \langle BE \rangle_{or} \langle BT \rangle$	$\{ \langle BE \rangle.FC = \langle BE \rangle_{or}.FC;$ $\langle BE \rangle.TC = \text{merge}(\langle BE \rangle_{or}.TC, \langle BT \rangle.TC) \}$

(5) (4) 构成and逻辑运算

(2) (1) 构成or逻辑运算

• 布尔表达式: $A \text{ and } B \text{ and } C > D$ 的四元式为:



(1) (jnz, A, —, 3)	}	/* A的四元式 */
(2) (j, —, —, 0)		
(3) (jnz, B, —, 5)	}	/* B的四元式 */
(4) (j, —, —, 2)		
(5) (j>, C, D, 0)	}	/* C>D的四元式 */
(6) (j, —, —, 4)		



控制语句的翻译

- 控制语句包括：
 - if 语句
 - While 语句
 - Dowhile语句
 - For 语句

六 IF语句的翻译

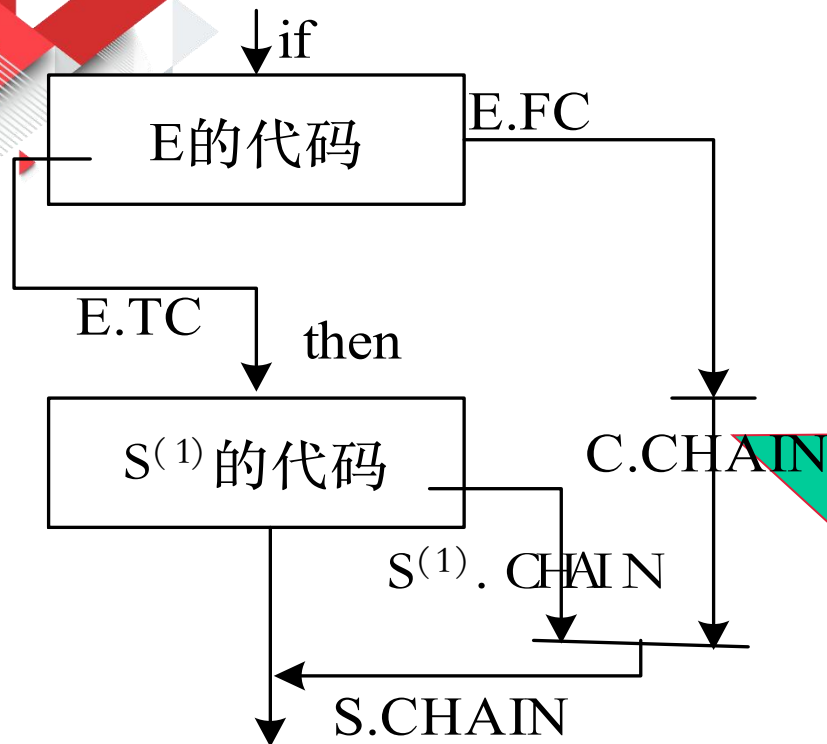
1. IF语句的文法(S是开始符号)

- (1) $S \rightarrow C S(1)$
- (2) $C \rightarrow \text{if } E \text{ then}$
- (3) $S \rightarrow T S(2)$
- (4) $T \rightarrow C S(1) \text{ else}$

- 产生式(1),(2)生成无else 的IF语句结构
- 产生式(2),(3),(4)生成if – then – else 的语句结构

2. IF语句的目标结构及其翻译 无else的结构

if a>b then x=3;



C.Chain的作用：由于在用第一个产生式进行归约时，只生成了条件式E的代码，**then**时可以回填E.TC, E.FC必须向后传递到下一个产生式中。

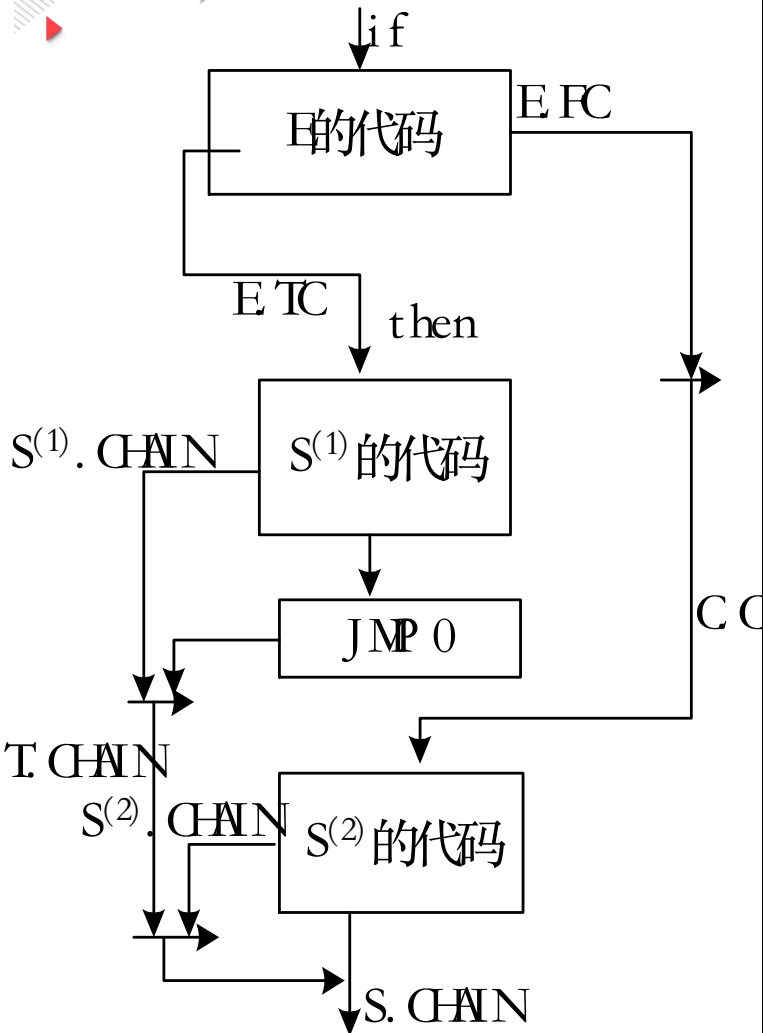
(1)	$S \rightarrow C S^{(1)}$	{ $S \cdot CHAIN = \text{MERG}(C \cdot CHAIN, S^{(1)} \cdot CHAIN)$ }
(2)	$C \rightarrow \text{if } E \text{ then}$	{ $\text{BACKPATCH}(E \cdot TC, NXQ);$ $C \cdot CHAIN = E \cdot FC$ }

if $a > b$ then $x := 3$
 else $x := 5$;



2. IF语句的目标结构及其翻译

– 有else的结构



语义规则

$C \rightarrow \text{if } E \text{ then}$

{ backpatch(E.TC, NXQ);
 $C.CHAIN = E.FC$ }

$S \rightarrow T \ S^{(2)}$

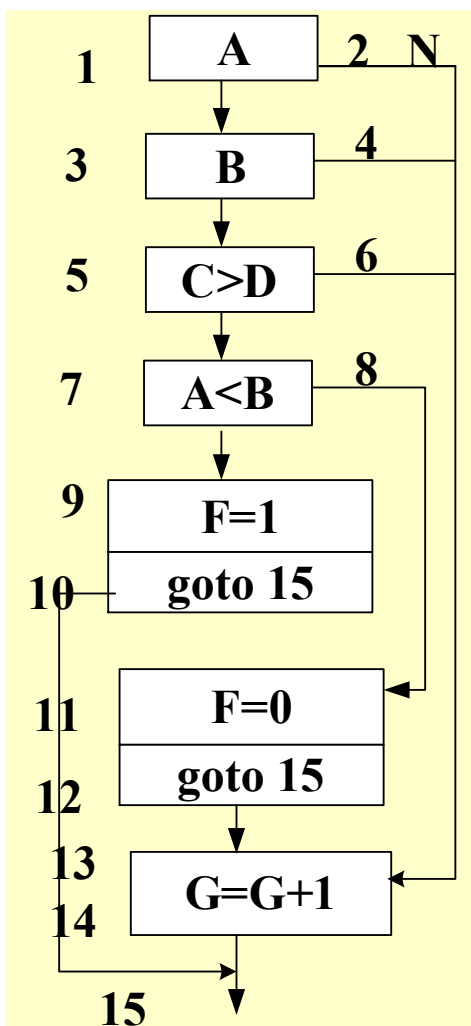
{ $S.CHAIN =$
 $\text{merge}(T.CHAIN, S^{(2)}.CHAIN)$ }

$T \rightarrow C \ S^{(1)} \text{ else}$

{ $q = NXQ$;
 $\text{gencode}(j, _, _, 0)$;
 $\text{backpatch}(C.CHAIN, NXQ)$;
 $T.CHAIN = \text{merge}(S^{(1)}.CHAIN, q)$ }

例：将下面的IF语句翻译为四元式序列

if A and B and (C>D) then
 if A<B then F=1
 else F=0
else G=G+1



- 1.(jnz,A,_,3) /*A的四元式*/
- 2.(j,_,_,13)
- 3.(jnz,B,_,5) /*B的四元式*/
- 4.(j,_,_,13)
- 5.(j>,C,D,7) /*C>D的四元式*/
- 6.(j,_,_,13)
- 7.(j<,A,B,9) /*A<B的四元式*/
- 8.(j,_,_,11)
- 9.(=,1,_,F) /*F :=1的四元式*/
- 10.(j,-,-,15)
- 11.(=,0,_,F) /*F :=0的四元式*/
- 12.(j,_,_,15)
- 13.(+,G,1,T) /*G:=G+1的四元式*/
- 14.(=,T,_,G)
- 15.

练习:将下面的语句翻译为四元式序列

if (A<C) and (B<D) then

if A=1 then

C=C+1

else if A≤D then

1. (j<,A,C,3)
A=A+2;

2. (j,-, -,14)

3. (j<,B,D,5)

4. (j,-, -,14)

5. (j=,A,1,7)

6. (j,-, -,10)

7. (+,C,1,T₁)

8. (=,T,-,C)

9. (j,-, -,14)

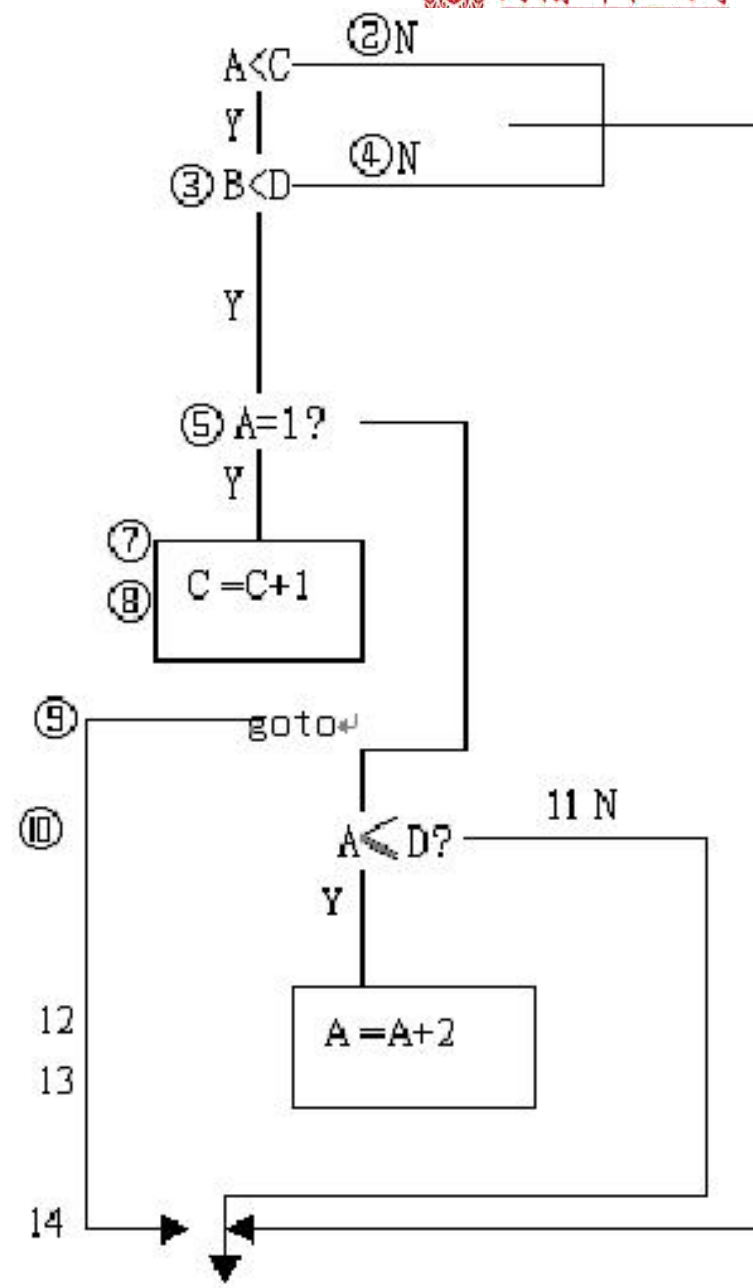
10. (j≤,A,D,12)

11. (j,-, -,14)

12. (+,A,2,T₂)

13. (=,T₂,-,A)

14.



Do while语句的翻译

1. 文法描述

$S \rightarrow \text{do } S^{(1)} \text{ while } E$

$D \rightarrow \text{do}$

$U \rightarrow DS^{(1)} \text{ while}$

$S \rightarrow UE$

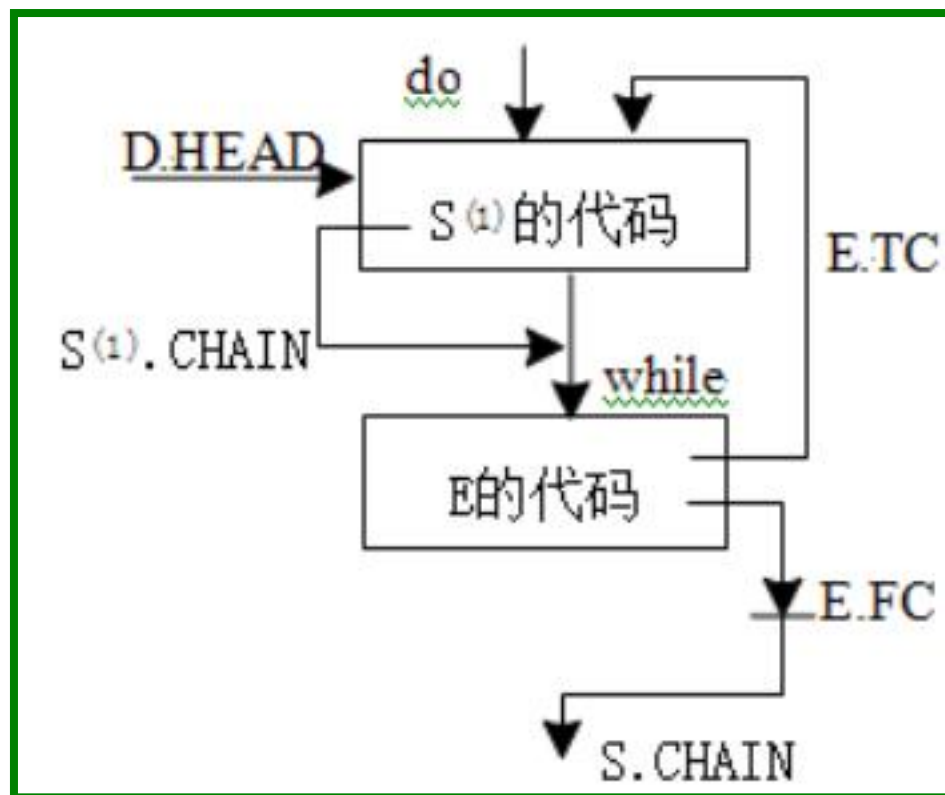
例:

do

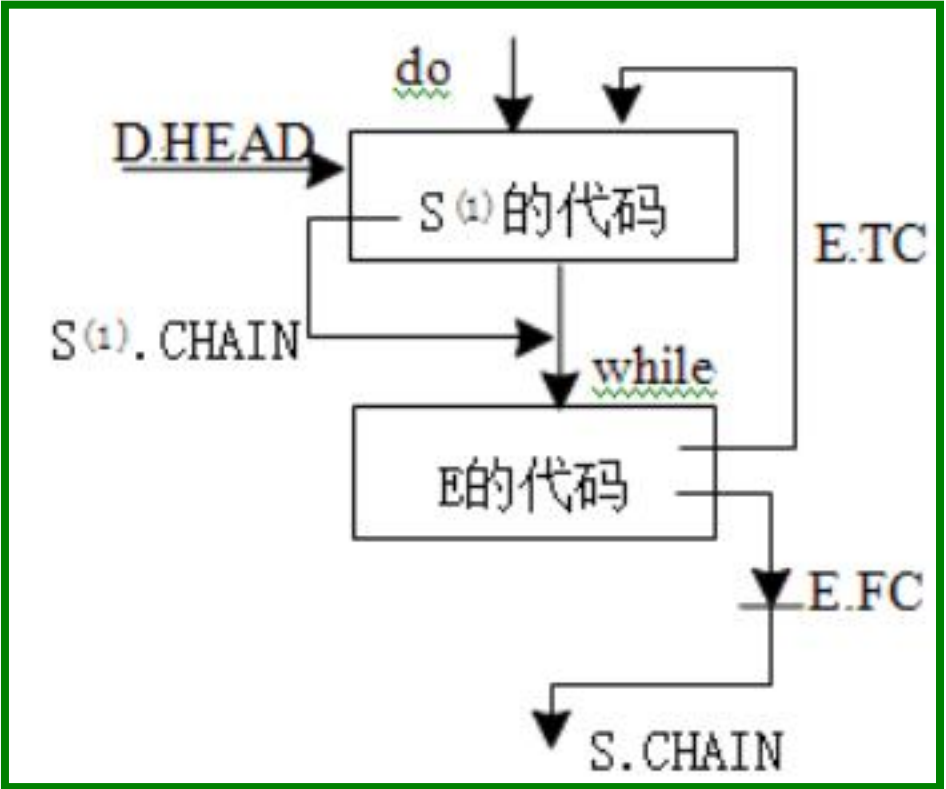
$x=3+a;$

while ($a>b$);

2. 目标结构



3. 翻译

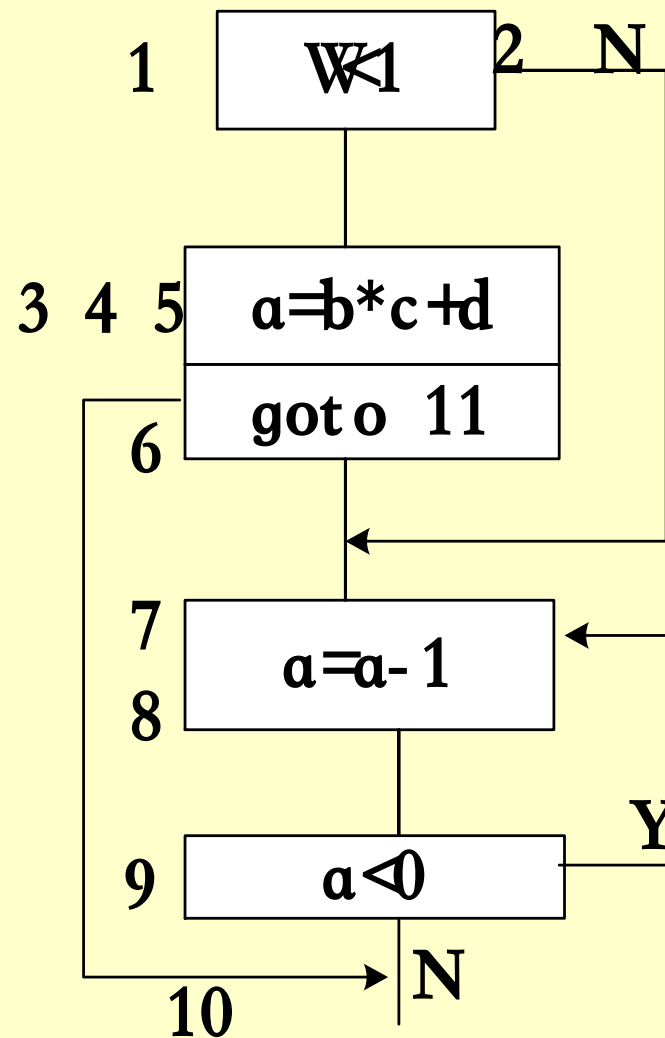


编号	产生式	语义规则
(1)	$D \rightarrow do$	{ $D.HEAD = NXQ$ }
(2)	$U \rightarrow DS^{(1)} while$	{ $U.HEAD = D.HEAD;$ $backpatch(S^{(1)}.CHAIN, NXQ)$ }
(3)	$S \rightarrow UE$	{ $backpatch(E.TC, U.HEAD);$ $S.CHAIN = E.FC$ }

将下面的语句翻译为四元式序列

- (1) $(j <, w, 1, 3)$
- (2) $(j, _, _, 7)$
- (3) $(*, b, c, T_1)$
- (4) $(+, T_1, d, T_2)$
- (5) $(=, T_2, _, a)$
- (6) $(j, _, _, 10)$
- (7) $(-, a, 1, T_3)$
- (8) $(=, T_3, _, a)$
- (9) $(j <, a, 0, 7)$
- (10)

if $w < 1$ then $a = b * c + d$
else do $a = a - 1$
while($a < 0$);



FOR语句的翻译

1. 文法描述

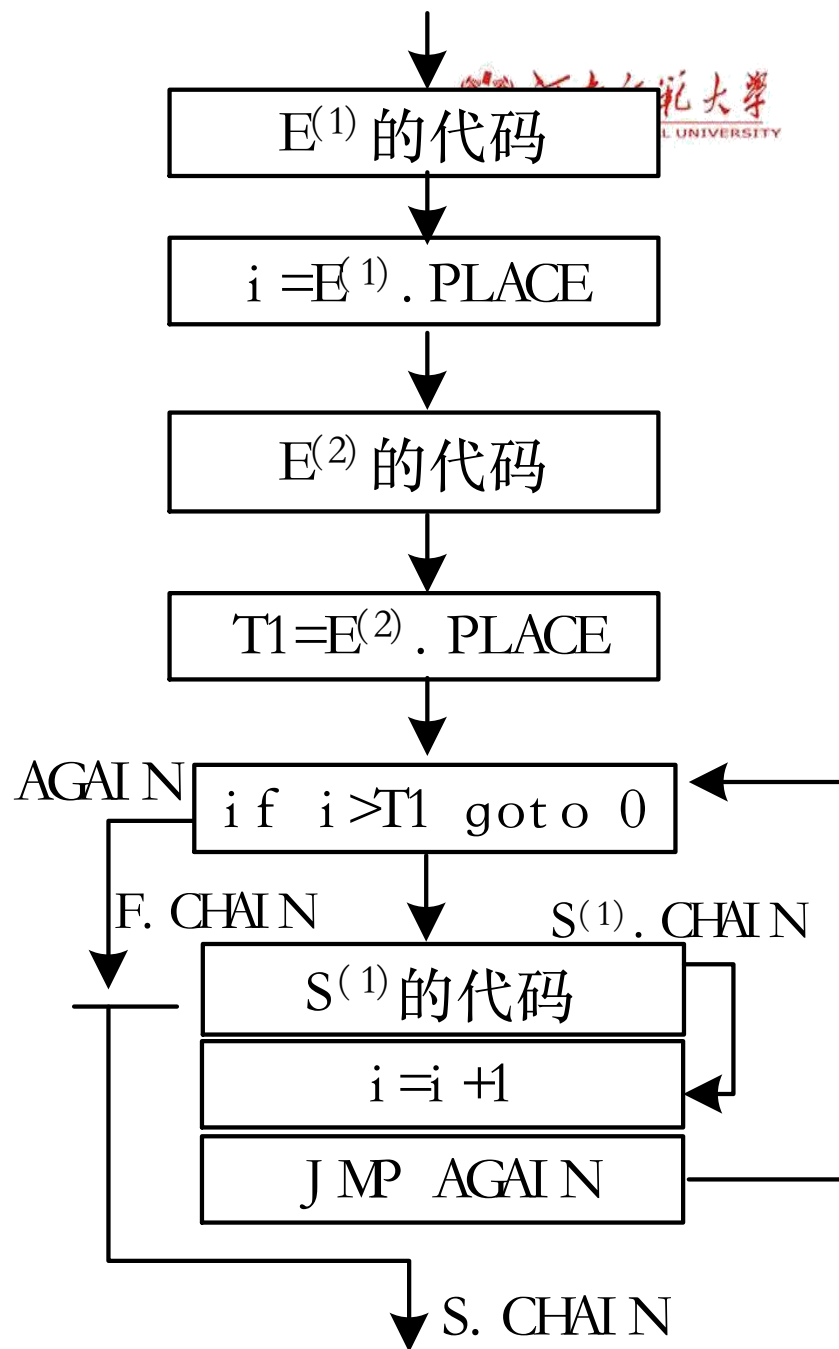
(1) $F \rightarrow \text{for } i = E^{(1)} \text{ to } E^{(2)} \text{ do}$

(2) $S \rightarrow FS^{(1)}$

例:

for i=1 to a+b do
x=x+1;

2. 目标结构



FOR语句的翻译

for i=1 to a+b do

x=x+1 河南师范大学
HENAN NORMAL UNIVERSITY

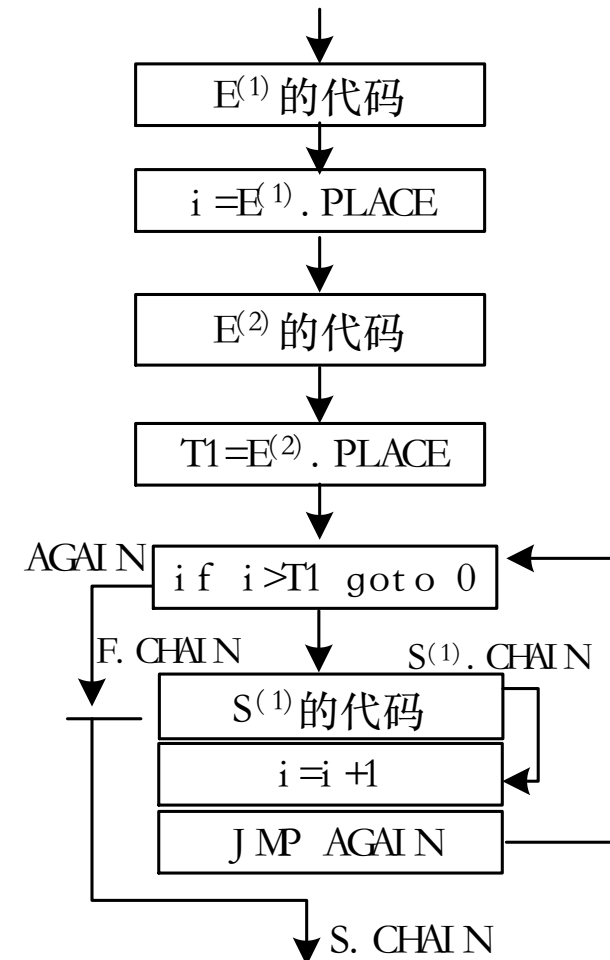
$F \rightarrow \text{for id} = E^{(1)} \text{ to } E^{(2)} \text{ do}$

```
{ F.PLACE = entry(id);  
  gencode(=, E(1).PLACE, _, F.PLACE);  
  T1 = newtemp(); /*T1用于存放循环终值*/  
  gencode(=, E(2).PLACE, _, T1);  
  F.AGAIN = NXQ;  
  F.CHAIN = NXQ;  
  gencode(j>, F.PLACE, T1, 0) }
```

$S \rightarrow F S^{(1)}$

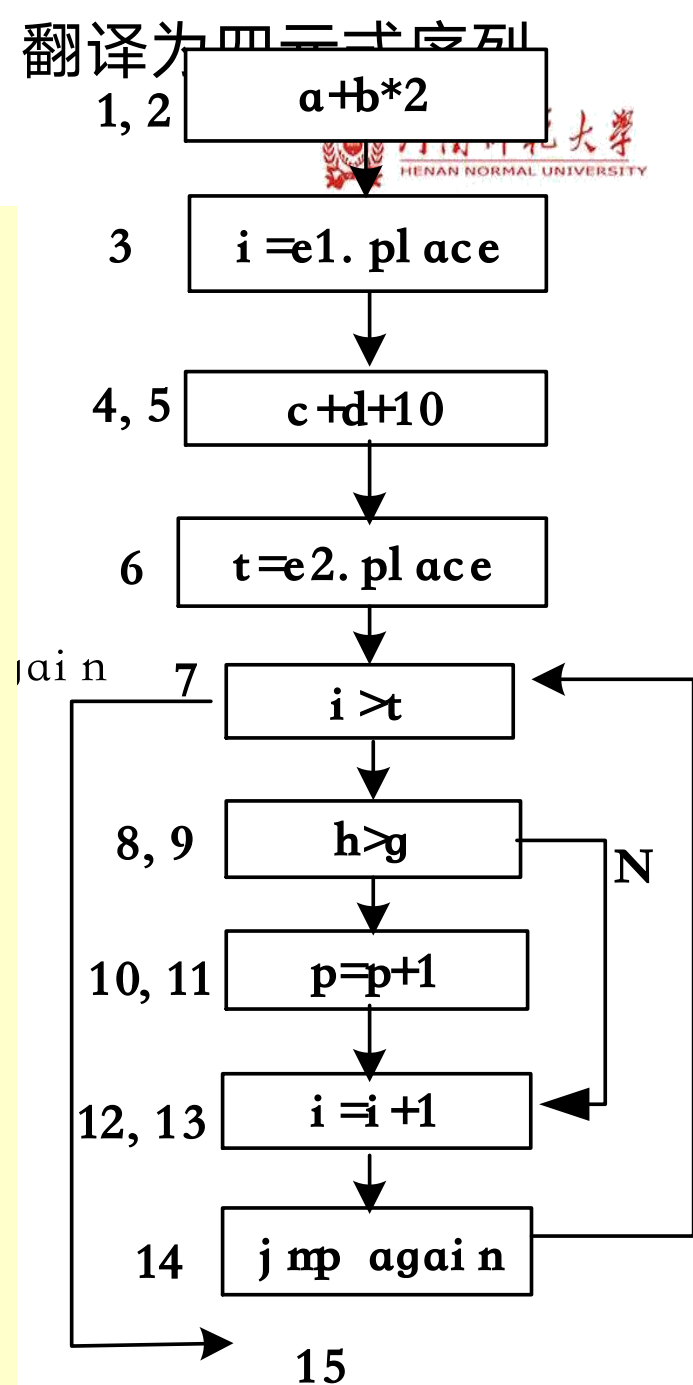
/*S是开始符号*/

```
{ backpatch(S(1).CHAIN, NXQ)  
  gencode(+, F.PLACE, 1, F.PLACE);  
  gencode(j, _, _, F.AGAIN);  
  S.CHAIN = F.CHAIN }
```



例: for i = a+b*2 to c+d+10 do 翻译为四三式序列
if h>g then p=p+1;

- (1) (*, b, 2, T₁)
- (2) (+, a, T₁, T₂)
- (3) (=, T₂, _, i)
- (4) (+, c, d, T₃)
- (5) (+, T₃, 10, T₄)
- (6) (=, T₄, _, t)
- (7) (j>, i, t, 15)
- (8) (j> h, g, 10)
- (9) (j, _, _, 12)
- (10) (+, p, 1, T₅)
- (11) (=, T₅, _, p)
- (12) (+, i, 1, T₆)
- (13) (=, T₆, _, i)
- (14) (j, _, _, 7)
- (15)





5.5 Sample语言语法制导翻译程序的设计



HENAN NORMAL UNIVERSITY

- 语法制导的翻译实际上就是在语法分析的基础上，当分析完一个正确的语法单位后，调用相应的语义处理程序，直接生成相应的四元式表。
- 在第4章使用递归下降的分析方法进行了Sample语言的语法分析器
- 在此基础上添加语义处理

举例：为IF语句添加语义处理

```
ifs( ) { /* If语句的语法分析*/  
    token = getnexttoken();  
    If(token!="if") error;  
    token= getnexttoken();  
    bexp();  
    token = getnexttoken();  
    If(token!="then") error;  
    token = getnexttoken();  
    ST_SORT();/*调用函数处理then后的可执行语句*/  
    token = getnexttoken();  
    If(token!= "else") error;  
    token = getnexttoken();  
    ST_SORT();/*处理else后的可执行语句*/  
}
```

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>

```
ifs( ) { /* 添加语义处理后的程序 */
```

```
token = getnexttoken();
```

```
if (token != "if") error;
```

```
token = getnexttoken();
```

```
(e.tc, e.fc) = bexp();
```

```
token = getnexttoken();
```

```
if (token != "then") error("缺then");
```

```
backpatch(e.tc, nxq); /*回填真出口e.tc*/
```

```
token = getnexttoken();
```

```
s1.chain = ST_SORT(token); /*处理s1,返回s1链*/
```

```
token = getnexttoken();
```

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>

```
if token = "else" /* 处理else部分*/
```

```
{ q = nxq;
```

```
gencode(j, —, —, 0);
```

```
backpatch(e.fc, nxq); /*回填假出口e.fc*/
```

```
t.chain = merg(s1.chain, q);
```

```
getnexttoken(token);
```

```
s2.chain = ST_SORT(token);
```

```
s.chain = merg(t.chain, s2.chain);
```

```
return(s.chain)
```

```
}
```

```
else /* if -- then结构时*/
```

```
return(merg(s1.chain, e.fc))
```

```
}
```

<if语句> ::= if <布尔表达式> then <执行句> else <执行句>



Sample语言语法制导翻译程序的总控程序的编写



- 语法制导翻译程序的总控程序既要负责处理调用各个语句的处理，又要负责翻译PROGRAM的产生式（ $\langle \text{程序} \rangle ::= \text{program } \langle \text{标识符} \rangle ; \langle \text{分程序} \rangle$ 和 $\langle \text{分程序} \rangle ::= \langle \text{常量说明} \rangle \langle \text{变量说明} \rangle \langle \text{复合句} \rangle .$ ）。
- 当匹配了” program”和程序名标识符id后，应生成四元式（program, id, $_$, $_$ ），这个四元式表示目标程序执行前的一些准备工作，这些工作通常和机器有关，包括包护一些寄存器，设置保护区，为用户程序分配一定的运行空间等。
- 当匹配到” end.”这两个符号时，则应生成四元式（sys, $_$, $_$, $_$ ），用于表示恢复寄存器，释放保留区，退出程序的运行状态等操作。

```
void paser_senmatic( ) /*语法制定翻译的总控程序*/
{
    token = getnexttoken();
    if (token != "program" ) error(); /*程序中缺少program*/
    token = getnexttoken();
    if (token 不是 标识符) error(); /*program后应跟程序名称*/
    gencode("program",token,_,_);

    .....
    token = getnexttoken();
    if (token 不是 'end.') error(); /*end.标识整个程序结束*/
    gencode("sys",_,_,_);
}
```

用C语言实现repeat()

```
int *repeats()/* repeat语句的语法制导的翻译过程*/
{
    r_head = nxq; /*记录repeat语句开头的四元式编号*/
    token= getnexttoken();/*读取下一个token字*/
    sl_chain = ST_SORT(token);/*处理repeat后的语句*/
    token= getnexttoken();
    if (!strcmp(token, ";")) /*读到;*/
        token= getnexttoken();
    if (strcmp(token, "until")) error("缺少 until");/*应为until*/
    backpatch(&sl_chain, &nxq);
    token= getnexttoken();
    bexp(&be_tc, &be_fc, token);/*处理布尔表达式*/
    backpatch(&be_fc, &r_head);/*回填假出口*/
    return &be_tc; /*返回真出口，即repeat语句的下一个语句的四元式编号*/
}
```